



Universidad
Tecmilenio®

NOMBRE: JOAQUIN RAUL NAJERA SALDAÑA

MATRICULA: 7132489

MAESTRO: Edgar Ivan Patricio Aizpuru

Sistema de Gestión de una Biblioteca mediante una Lista Simplemente Enlazada

Introducción

Las estructuras de datos son fundamentales en el desarrollo de software, ya que permiten organizar, almacenar y manipular información de manera eficiente. Entre estas estructuras se encuentran las listas enlazadas, las cuales están formadas por nodos conectados mediante referencias.

En esta actividad se desarrolló un sistema de gestión de una biblioteca utilizando el lenguaje de programación Java y una lista simplemente enlazada implementada de manera propia. El objetivo principal fue aplicar los conceptos estudiados durante la semana, como nodos, referencias, head, next, recorridos, inserciones, búsquedas, eliminaciones y acceso mediante posiciones.

El programa permite administrar libros registrados en una biblioteca, almacenando información como título, autor y código identificador. Además, permite realizar diferentes operaciones sobre los libros, como agregarlos, buscarlos, consultarlos, eliminarlos y conocer la cantidad total de elementos almacenados.

Desarrollo

1. Estructura del nodo

La primera parte del programa consiste en crear la clase Node. Esta clase representa cada elemento individual de la lista enlazada.

Cada nodo contiene los datos correspondientes a un libro:

- Título.
- Autor.
- Código identificador.
- Referencia al siguiente nodo.

La referencia next es la encargada de conectar un nodo con el siguiente. Cuando un nodo es el último de la lista, su referencia next contiene el valor null.

La estructura puede representarse de la siguiente manera:

[Libro A] → [Libro B] → [Libro C] → null

De esta forma, cada nodo conoce cuál es el nodo que se encuentra después de él.

2. Referencia head

La clase LinkedList utiliza una referencia llamada head. Esta referencia representa el primer nodo de la lista.

Cuando la lista está vacía:

head → null

Cuando existen varios libros:

head

↓

[Libro A] → [Libro B] → [Libro C] → null

El uso de head permite acceder al primer elemento y, a partir de él, recorrer todos los nodos de la lista.

3. Recorrido de la lista

Para recorrer la lista se utiliza una referencia auxiliar llamada current.

Por ejemplo:

Node current = head;

```
while (current != null) {  
    System.out.println(current);  
    current = current.next;  
}
```

Primero, current apunta al primer nodo. Posteriormente, mediante current.next, avanza al siguiente nodo hasta llegar a null.

Este procedimiento permite mostrar todos los libros registrados sin modificar la referencia head.

4. Inserción al inicio

Una de las operaciones implementadas es insertar un libro al principio de la lista.

Si inicialmente tenemos:

$$A \rightarrow B \rightarrow C$$

y agregamos el libro X al inicio, el nuevo nodo apunta primero al antiguo head:

$$X \rightarrow A \rightarrow B \rightarrow C$$

Después, head se actualiza para apuntar al nuevo nodo:

head

↓

$$X \rightarrow A \rightarrow B \rightarrow C \rightarrow \text{null}$$

Esta operación permite agregar un elemento sin necesidad de recorrer toda la lista.

5. Inserción al final

También se implementó la inserción de un libro al final.

Para realizarla se recorre la lista utilizando current hasta encontrar el último nodo, cuyo next apunta a null.

Por ejemplo:

$$A \rightarrow B \rightarrow C \rightarrow \text{null}$$

Después de insertar X:

$$A \rightarrow B \rightarrow C \rightarrow X \rightarrow \text{null}$$

Para realizar esta operación es necesario modificar la referencia next del último nodo.

6. Inserción en una posición determinada

El programa también permite insertar un libro en una posición específica. Las posiciones comienzan desde cero.

Por ejemplo:

Posición:

0 1 2

$$A \rightarrow B \rightarrow D$$

Si se inserta C en la posición 2:

$A \rightarrow B \rightarrow C \rightarrow D$

Para realizar esta operación se localiza el nodo anterior a la posición deseada y se modifican las referencias correspondientes.

7. Búsqueda de libros

La búsqueda permite localizar un libro utilizando su código identificador.

El programa recorre los nodos desde head hasta encontrar un código coincidente.

Si el libro existe, se muestran sus datos. Si no existe, el programa informa al usuario que no se encontró ningún libro con ese código.

Esto permite comprobar el funcionamiento de los recorridos y las comparaciones dentro de la lista.

8. Consulta por posición

Otra operación implementada es obtener un libro utilizando su posición.

Por ejemplo:

0 $\rightarrow$ Libro A

1 $\rightarrow$ Libro B

2 $\rightarrow$ Libro C

Si el usuario solicita la posición 1, el programa devuelve el Libro B.

También se validan posiciones incorrectas, como valores negativos o posiciones superiores al tamaño de la lista.

9. Eliminación de elementos

La eliminación es una de las operaciones más importantes de una lista enlazada, debido a que requiere modificar referencias entre los nodos.

Para eliminar el primer elemento, simplemente se actualiza head:

`head = head.next;`

Por ejemplo:

Antes:

head

↓

$A \rightarrow B \rightarrow C$

Después:

head

↓

$B \rightarrow C$

Para eliminar un nodo intermedio se modifica la referencia del nodo anterior.

Por ejemplo:

$A \rightarrow B \rightarrow C \rightarrow D$

Si se desea eliminar C, se puede realizar:

`current.next = current.next.next;`

El resultado es:

$A \rightarrow B \rightarrow D$

La operación hace que B deje de apuntar a C y pase a apuntar directamente a D.

También se contempla la eliminación del último elemento y el intento de eliminar libros que no existen.

10. Tamaño de la lista

El programa cuenta la cantidad de libros recorriendo todos los nodos.

Por ejemplo:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{null}$

El resultado del tamaño es:

4

El contador comienza en cero y aumenta cada vez que se encuentra un nodo.

11. Manejo de casos especiales

Para evitar errores durante la ejecución se contemplaron diferentes situaciones:

- Lista vacía.

- Lista con un solo libro.
- Posiciones negativas.
- Posiciones que no existen.
- Búsqueda de un libro inexistente.
- Eliminación de un libro inexistente.
- Entrada de datos inválida.

Por ejemplo, si la lista está vacía y el usuario intenta mostrar los libros, el programa informa que no existen elementos en lugar de producir un error.

También se implementó validación para las entradas numéricas del menú.

12. Programa principal

La clase Main funciona como punto de entrada del programa y proporciona un menú para que el usuario pueda interactuar con la biblioteca.

El menú permite realizar las siguientes operaciones:

1. Agregar libro al inicio
2. Agregar libro al final
3. Insertar libro en posición
4. Mostrar libros
5. Buscar libro
6. Consultar libro por posición
7. Eliminar libro
8. Eliminar libro por posición
9. Mostrar cantidad de libros
10. Salir

De esta manera, todas las operaciones de la lista pueden probarse directamente desde la consola.

Pruebas realizadas

Para comprobar el funcionamiento del programa se deben realizar diferentes pruebas, incluyendo casos normales y casos límite.

Prueba	Entrada	Resultado esperado
Lista vacía	Mostrar libros	Informar que la lista está vacía
Agregar al inicio	Libro A	Libro A se convierte en head
Agregar al final	Libro B	Libro B queda al final
Insertar en posición	Libro C, posición 1	Libro C se inserta correctamente
Buscar existente	Código de Libro A	Libro encontrado
Buscar inexistente	Código Z999	Libro no encontrado
Consultar posición válida	Posición 1	Se muestra el libro correspondiente
Consultar posición inválida	Posición 100	Se informa que la posición no existe
Eliminar primero	Posición 0	head cambia al siguiente nodo
Eliminar intermedio	Posición 1	El nodo es eliminado y los enlaces se mantienen
Eliminar último	Última posición	El último nodo es eliminado
Eliminar inexistente	Código Z999	Se informa que no existe
Obtener tamaño	Lista con varios libros	Se muestra la cantidad correcta

Entrada inválida	Texto donde se espera número	Se solicita nuevamente un número
------------------	------------------------------	----------------------------------

Estas pruebas permiten comprobar que la estructura funciona correctamente en diferentes situaciones y que los casos especiales son manejados sin detener el programa.

Conclusión

La realización de este proyecto permitió aplicar los conceptos fundamentales de las listas simplemente enlazadas mediante un problema práctico: la administración de libros de una biblioteca.

Durante la implementación se trabajó con conceptos como Node, head, next y referencias auxiliares como current. También se comprendió cómo los nodos se conectan entre sí y cómo las operaciones de inserción y eliminación modifican dichas referencias.

Una de las partes más importantes fue comprender que una lista enlazada no almacena los elementos de la misma manera que un arreglo. En una lista simplemente enlazada, cada nodo mantiene una referencia hacia el siguiente elemento, lo que permite insertar y eliminar nodos modificando las conexiones entre ellos.

Finalmente, las pruebas realizadas permiten verificar que el programa puede manejar operaciones normales y situaciones especiales, como listas vacías, posiciones inválidas y elementos inexistentes. Con este proyecto se fortaleció la comprensión de las estructuras de datos y de la forma en que pueden utilizarse para resolver problemas reales mediante programación en Java.